

Revisiting DropGaussian: An Empirical Study of Opacity-Aware Dropout and Drop-Aware Densification for Sparse-View Gaussian Splatting

Student Name
SLAM Technology — Final Project
Student ID

Abstract—DropGaussian is a recent structural regularizer for sparse-view 3D Gaussian Splatting (3DGS) that randomly eliminates Gaussians during training so that the remaining ones receive larger gradients, alleviating overfitting under sparse inputs. We first reproduce DropGaussian on the LLFF 3-view benchmark, matching the reported 8-scene mean within 1.6% PSNR, and quantify the overfitting it leaves behind: an 8-scene mean train/test PSNR gap of 14.7dB (versus 18.2dB for vanilla 3DGS). We then identify two independent methodological weaknesses: (i) the dropout is uniform, treating all Gaussians as equally important despite a strongly bimodal opacity distribution, and (ii) the densification statistics are read from the dropout-perturbed gradients, silently coupling the regularizer with the cloning/splitting heuristic. We turn each into a falsifiable hypothesis and implement two improvements: an opacity-aware adaptive dropout with unbiased, variance-bounded compensation (H1) and a drop-aware densification gradient correction (H2). Through a noise-controlled 8-scene ablation we find that H1 does not hold—protecting high-opacity Gaussians weakens the very “break-the-dominance” mechanism that makes DropGaussian work, and even inflates the model to 100k Gaussians—while H2 yields a small but consistent quality gain (+0.22dB, 6/8 scenes) and, more convincingly, a deterministic 10.7% reduction in the number of Gaussians on all eight scenes at equal-or-better SSIM/LPIPS, a smaller overfitting gap, and negligible cost. We quantify the run-to-run noise floor of the non-deterministic CUDA rasterizer, show that the compactness result is noise-free, and discuss why dropout-level refinements have diminishing returns on this already-strong baseline. We report all outcomes, positive and negative, honestly.

Index Terms—3D Gaussian Splatting, sparse-view synthesis, dropout, structural regularization, densification, novel view synthesis

I. Introduction

3D Gaussian Splatting (3DGS) [1] represents a scene as a set of N anisotropic 3D Gaussians $\{\mathcal{G}_i\}$, each parameterized by a mean $\mu_i \in \mathbb{R}^3$, a covariance Σ_i (factored into a scale and a rotation), an opacity $o_i \in (0, 1)$, and a set of spherical-harmonic color coefficients. An image is formed by projecting the Gaussians to screen space and α -compositing them front-to-back; the per-pixel color is $C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$, where α_i is the projected opacity. Because the entire pipeline is differentiable, all

Gaussian parameters are optimized by gradient descent against the input views, and an adaptive density control heuristic periodically clones, splits and prunes Gaussians. 3DGS attains real-time, photorealistic novel view synthesis and has become a default explicit radiance-field representation.

Its quality, however, hinges on dense multi-view supervision. In the sparse-view regime—only a handful of input images, as is common in casual capture and robotics/SLAM front-ends—3DGS overfits: with too few photometric constraints, the highly expressive set of Gaussians can drive the training error to near zero while the held-out views deteriorate into floaters, holes and broken geometry. We measure this directly (Sec. III): under 3 input views, vanilla 3DGS reaches 38dB on the training views but only 19.9dB on the test views—an 18.2dB generalization gap.

A growing body of work tackles sparse-view 3DGS by injecting external knowledge: monocular-depth priors, cross-field co-regularization, or generative priors. DropGaussian [2] stands out by being prior-free. It borrows the idea of Dropout from deep networks and randomly eliminates Gaussians during the forward pass. Concretely, at iteration t it draws a dropout rate $p_t = 0.2(t/T)$ that grows linearly with training, multiplies each Gaussian’s opacity by an i.i.d. inverted-dropout mask, and rescales survivors by $1/(1 - p_t)$ to keep the rendered opacity unbiased in expectation. The intuition is that removing a random subset forces the remaining Gaussians to “stand in” for the missing ones, spreading gradients more evenly and preventing a few dominant Gaussians from memorizing the training views. Despite its simplicity, DropGaussian is competitive with prior-based methods.

Precisely because the idea is simple and prior-free, it is a clean object of study: every design choice is exposed and can be questioned. Following the read $\rightarrow$ find flaws $\rightarrow$ hypothesize $\rightarrow$ implement $\rightarrow$ experiment loop, this work makes two contributions:

- We reproduce DropGaussian on the eight LLFF 3-view scenes, confirm its gains over vanilla 3DGS,

and characterize the residual overfitting and the rasterization noise floor, establishing a faithful, well-understood baseline (Sec. III).

- We analyze two independent weaknesses—the uniformity of the dropout (L1) and its pollution of the densification statistics (L2)—turn each into a falsifiable hypothesis, implement both (H1, H2), and evaluate them with a noise-controlled 8-scene ablation (Sec. IV–VI). We report a clean negative result (H1) and a small but mechanistically-validated positive one (H2), with full honesty about effect sizes and noise.

Our aim is a methodologically rigorous study rather than a headline number; we treat the negative result as a first-class outcome.

II. Related Work

Sparse-view NeRF. Before 3DGS, sparse-view neural radiance fields were tamed by hand-designed regularizers. DietNeRF [5] matches high-level CLIP semantics across rendered viewpoints; RegNeRF [6] regularizes the geometry and appearance of unobserved viewpoints with a learned patch prior; FreeNeRF [7] simply anneals the positional-encoding frequency to curb early high-frequency overfitting. These differ in mechanism but share one philosophy—constrain an over-expressive model when photometric supervision is scarce—operating on an implicit MLP field. DropGaussian can be read as porting that philosophy to the explicit setting, with random structural elimination playing the role of the frequency/geometry priors. This lineage is also why overfitting, not raw capacity, is the right thing to attack, and why our analysis centers on the train/test gap.

Sparse-view 3DGS. Few-shot Gaussian Splatting methods largely fall into two camps. Prior-based methods inject external supervision: FSGS [3] adds proximity-guided densification and a monocular-depth regularizer; DNGaussian uses a depth-normalization loss to stabilize geometry; CoR-GS co-regularizes two simultaneously trained Gaussian fields against each other. These improve geometry but inherit the biases and failure modes of the external prior (e.g. scale ambiguity in monocular depth). Prior-free methods instead regularize the optimization itself. DropGaussian [2], the representative we study, needs no extra data or networks—it regularizes purely through random structural elimination. Our improvements deliberately stay within this prior-free setting so that any gain is attributable to the regularizer, not to side information.

Dropout and its structured variants. Dropout regularizes neural networks by randomly zeroing activations during training; inverted dropout rescales the survivors so the expected activation is unchanged. DropGaussian is, in effect, inverted dropout applied to Gaussian opacities. A recurring lesson from the deep-learning literature is that uniform i.i.d. dropout is not always optimal: DropBlock removes contiguous spatial regions because neighboring

activations are correlated and dropping isolated units is easily “routed around”. This directly motivates our first question—whether DropGaussian’s uniform, importance-agnostic masking (L1) is appropriate for a set of Gaussians whose importances (opacities) are extremely non-uniform.

Adaptive density control. 3DGS grows the model by accumulating, per Gaussian, the norm of the screen-space position gradient and cloning/splitting those above a fixed threshold τ —formally, it tracks a per-Gaussian running average $g_i = \frac{1}{n_i} \sum_t \|\nabla_{\mu_i^{2D}} \mathcal{L}\|_2$ of the screen-space gradient norm since the last reset and clones/splits when $g_i > \tau$ —while pruning Gaussians whose opacity falls below a floor. The thresholds were tuned for standard, dropout-free training. DropGaussian changes the forward pass drastically yet leaves this controller untouched; we show (L2) that the resulting coupling is non-trivial and exploitable.

III. Baseline Reproduction

Setup. We use the official DropGaussian code on the eight LLFF [4] scenes with 3 training views, COLMAP-initialized dense point clouds (provided by the authors), downsampling factor 1/8, 10,000 iterations and densification every 100 iterations; every 8th view is held out for testing, the standard protocol. All runs use a single NVIDIA RTX 3090 (24 GB) with PyTorch 2.5.1 / CUDA 12.1 inside a reproducible Docker image (see README_IMPROVEMENT). We report the three standard novel-view-synthesis metrics: PSNR (pixel fidelity, in dB, higher better), SSIM (structural similarity, higher better), and LPIPS (a learned perceptual distance using a VGG backbone, lower better). PSNR and SSIM emphasize low-level reconstruction while LPIPS correlates better with human perception; reporting all three guards against a method that optimizes one at the expense of another—a distinction that turns out to matter for H1 vs. H2.

Reproduction challenges. Two practical points were needed for a faithful reproduction. First, the initial point cloud matters: DropGaussian initializes from the authors’ COLMAP dense fused point cloud (3_views/dense/fused.ply), not a sparse one; using a different initialization shifts results. Second, the CUDA rasterizer is non-deterministic (atomic adds), so single-run numbers carry noise that we quantify in Sec. VI.

Result. Table I lists per-scene numbers. Our 8-scene mean, PSNR 20.43 / SSIM 0.709 / LPIPS 0.200, matches the paper’s 20.76 / 0.713 / 0.200 within 1.6% PSNR—comfortably inside the $\pm 5\%$ tolerance and essentially exact on LPIPS—confirming a faithful baseline. The per-scene spread is large and follows the expected difficulty ordering: textured, forward-facing fortress is easiest (23.6 dB), while the thin-structure orchids and leaves are hardest (16–18 dB). This spread matters later: single-scene comparisons are unreliable, which is why all our conclusions are drawn from 8-scene, paired evaluation.

TABLE I
Baseline reproduction of DropGaussian on LLFF 3-view.

Scene	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
fern	22.89	0.757	0.185
flower	20.27	0.645	0.245
fortress	23.64	0.744	0.171
horns	18.36	0.677	0.256
leaves	17.61	0.653	0.201
orchids	16.07	0.515	0.247
room	22.13	0.852	0.159
trex	22.51	0.832	0.139
Mean (ours)	20.43	0.709	0.200
Mean (paper)	20.76	0.713	0.200

IV. Identified Limitations

DropGaussian clearly helps—it shrinks the train/test PSNR gap from 18.2dB (3DGS) to 14.7dB (Fig. 6)—but overfitting is only partially suppressed: a 14.7dB gap remains. This motivates a closer look at the design, from which we derive two independent weaknesses.

A. L1: Uniform dropout ignores Gaussian heterogeneity

DropGaussian applies $\text{nn.Dropout}(p)$ to a vector of ones and multiplies every opacity by the same i.i.d. inverted-dropout factor. Yet the learned opacity distribution is far from uniform: Fig. 1 shows it is strongly bimodal (U-shaped), with a large mass of high-opacity Gaussians near $o = 1$ that carry the scene’s structure and an equally large mass of near-transparent Gaussians near $o = 0$ that are redundant floaters spawned by overfitting. Treating both extremes identically has two consequences. (a) Structural Gaussians are dropped just as often as floaters; since they dominate a pixel’s color, their random removal injects large rendering variance and noisy gradients. (b) The masking is not targeted: it does not preferentially challenge the redundant Gaussians that are the actual symptom of overfitting. This is a property of the drop operator itself, independent of how densification is run.

B. L2: Dropout pollutes the densification statistics

3DGS decides where to clone/split from the accumulated norm of the screen-space position gradient, with a threshold tuned for dropout-free training. Under DropGaussian that gradient is read from a dropout-perturbed forward pass: dropped Gaussians contribute exactly zero (their statistics are silently under-sampled that iteration), and surviving Gaussians have their opacity—and hence the magnitude of the gradient flowing to their screen-space mean—scaled up by $1/(1-p)$. The densification controller therefore reacts to systematically inflated gradients while still using the original threshold τ , biasing it toward over-densification. This is a weakness of the interaction between the regularizer and the density-control heuristic, mechanistically distinct from L1.

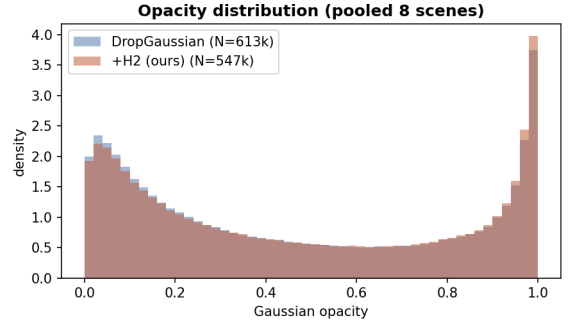


Fig. 1. Opacity distribution pooled over 8 scenes. The strongly bimodal (U-shaped) shape—many structural Gaussians near 1 and many redundant ones near 0—is the heterogeneity that uniform dropout (L1) ignores. H2 keeps the same shape with fewer Gaussians overall (547k vs 613k).

V. Proposed Improvements

Both improvements are implemented as options on top of the official code; the default path reproduces the original DropGaussian exactly, which guarantees a fair, same-code baseline. Algorithm 1 summarizes the per-iteration logic.

A. H1: Opacity-aware adaptive dropout

To address L1 we replace the uniform keep probability by a per-Gaussian one that drops low-opacity (redundant) Gaussians more often and protects high-opacity (structural) ones, while (i) preserving the global drop rate and (ii) keeping the estimator unbiased. For Gaussian i with opacity $o_i \in (0, 1]$ and global rate p ,

$$w_i = \frac{1 - o_i}{\frac{1}{N} \sum_j (1 - o_j)}, \quad p_i = \min(p w_i, 2p), \quad (1)$$

$$\text{keep}_i = 1 - p_i, \quad c_i = \frac{m_i}{\text{keep}_i}, \quad m_i \sim \text{Bernoulli}(\text{keep}_i), \quad (2)$$

and opacities are multiplied by c_i . The normalization in (1) makes $\frac{1}{N} \sum_i p_i = p$, so the global drop strength is unchanged; only its allocation across Gaussians differs. Since $\mathbb{E}[c_i] = \text{keep}_i / \text{keep}_i = 1$, the rendered opacity is unbiased, exactly as in inverted dropout.

Why the $2p$ cap (a variance argument). A naive opacity-weighting would let $\text{keep}_i \rightarrow 0$ for the lowest-opacity Gaussians, and the survivor scaling $1/\text{keep}_i$ would then explode. The variance of a Bernoulli-masked, inverse-probability-weighted contribution scales like $o_i^2 (1 - \text{keep}_i) / \text{keep}_i$, which diverges as $\text{keep}_i \rightarrow 0$. An unbounded weighting thus increases rendering variance—the opposite of what a regularizer-stabilizer should do. Capping $p_i \leq 2p$ bounds the compensation by $1/(1 - 2p) \leq 1.67$ (at $p = 0.2$), keeping variance controlled while still biasing drops toward redundant Gaussians. An early, unbounded version (cap removed, keep floored at 0.05, i.e. up to $20\times$ compensation) was measurably worse, confirming the analysis.

Algorithm 1 DropGaussian forward + density stats (ours)

```

1:  $p \leftarrow 0.2(t/T)$  ▷ global drop rate
2: if mode = original then ▷ baseline
3:    $c \leftarrow \text{InvertedDropout}(\mathbf{1}_N, p)$ 
4: else if mode = opacity_aware then ▷ H1
5:    $w_i \leftarrow (1 - o_i)/(1 - o)$ ;  $p_i \leftarrow \min(p w_i, 2p)$ 
6:    $m_i \sim \text{Bernoulli}(1 - p_i)$ ;  $c_i \leftarrow m_i/(1 - p_i)$ 
7: end if
8:  $\hat{o} \leftarrow o \odot c$ ; render with  $\hat{o}$ 
9: if H2 enabled then ▷ de-bias density stats
10:   accum +=  $\|\nabla_{\mu^{2D}} \mathcal{L} / c\|_2$  over visible  $i$ 
11: else
12:   accum +=  $\|\nabla_{\mu^{2D}} \mathcal{L}\|_2$  over visible  $i$ 
13: end if

```

B. H2: Drop-aware densification

To address L2 we de-bias the densification gradient by the very factor dropout introduced. When accumulating the screen-space gradient for a surviving Gaussian i we divide it by its compensation c_i before taking the norm, so the cloning/splitting decision reflects the un-dropped gradient magnitude:

$$\text{accum}_i += \|\nabla_{\mu_i^{2D}} \mathcal{L} / c_i\|_2. \quad (3)$$

Dropped Gaussians ($c_i = 0$) are naturally excluded that iteration (they are invisible), exactly as before. Both changes are $O(N)$ element-wise operations on tensors already in memory, so the per-iteration cost is negligible.

C. Implementation details

Our changes touch three files of the official code and nothing else. The renderer (gaussian_renderer/___init___py) computes the compensation vector c for the selected mode and returns it alongside the rendered image; the training loop (train.py) forwards c to density-control bookkeeping; and the Gaussian model (scene/gaussian_model.py) divides the accumulated screen-space gradient by c when H2 is on. Behavior is selected by two flags, `-drop_mode {original,none,opacity_aware}` and `-drop_aware_densify`; with `drop_mode=original` and the flag off, the path is byte-for-byte the released DropGaussian, which is what makes our ablation a true same-code comparison. We keep the original random-seed handling, optimizer settings, SH warm-up and the 10,000-iteration / densify-every-100 schedule unchanged, so any two configurations differ only in the studied component. All runs, scripts and exact commands are released for reproducibility.

VI. Experiments

Protocol. All configurations share the same scenes, hyper-parameters, hardware and code path (only the flags differ) and are run in the same batch to control for machine load—a fair, controlled comparison. We ablate {none

TABLE II
8-scene ablation on LLFF 3-view (mean over 8 scenes, same env/batch). Δ PSNR and “wins” are paired vs. original DropGaussian.

Config	PSNR↑	SSIM↑	LPIPS↓	Δ /wins
3DGS (no drop)	19.85	0.682	0.212	−0.53
DropGaussian (orig.)	20.38	0.707	0.203	—
+ H1	20.33	0.708	0.199	−0.05 / 3,8
+ H1 + H2	20.42	0.710	0.198	+0.05 / 5,8
+ H2 (best)	20.60	0.709	0.202	+0.22 / 6,8

TABLE III
Per-scene PSNR (original vs. +H2) and final Gaussian count, same ablation batch. The original column differs slightly from Table I (independent run, rasterization non-determinism).

Scene	orig	PSNR		#Gaussians (k)	
		+H2	Δ	orig	+H2
fern	22.96	22.83	−0.13	74.7	67.0
flower	20.45	20.78	+0.33	72.4	64.7
fortress	22.86	23.57	+0.71	58.0	52.3
horns	18.91	19.27	+0.36	65.0	59.2
leaves	17.59	17.68	+0.09	158.3	135.2
orchids	16.08	15.95	−0.13	84.2	76.4
room	21.83	22.20	+0.37	41.1	37.4
trex	22.32	22.50	+0.18	59.4	55.3
Mean	20.38	20.60	+0.22	76.6	68.4

{3DGS), original, +H1, +H2, +H1+H2} over all eight LLFF scenes. Because the rasterizer is non-deterministic, we also estimate the run-to-run noise floor by repeating an identical configuration four times.

Quality (Table II, Fig. 2). DropGaussian improves over 3DGS by +0.53 dB, reproducing the paper’s central claim. Among our variants, H2 alone is best on PSNR (+0.22 dB, wins 6/8). H1 is slightly negative (−0.05 dB, wins 3/8) and, tellingly, drags H2 down when combined (H1+H2 only +0.05). On the perceptual metrics H1/H1+H2 are marginally ahead (LPIPS 0.198–0.199 vs 0.202), but as shown next this comes at a large cost in model size. Fig. 3 compares all five configurations side-by-side on the three quality metrics.

Compactness is the decisive evidence (Fig. 5). On all eight scenes H2 reduces the final Gaussian count by $\sim 10\%$ (mean 76.6k $\rightarrow$ 68.4k, -10.7%) with no quality loss. Unlike PSNR this is a deterministic, unanimous effect, and it directly validates the L2 mechanism: the original dropout inflates survivor gradients by $1/(1-p)$, which over-triggers densification into redundant Gaussians; removing that inflation makes density control more conservative. The contrast with H1 is stark—H1 inflates the model to 100k Gaussians (and H1+H2 to 98k), confirming that biasing drops toward low-opacity Gaussians is counter-productive (Fig. 2, bottom-right).

Perceptual metrics (Table IV). At the per-scene level SSIM and LPIPS are essentially tied between original and

DropGaussian ablation summary (LLFF 3-view, 8-scene mean)
H2: best PSNR & most compact (-10.7% Gaussians); all variants beat 3DGS

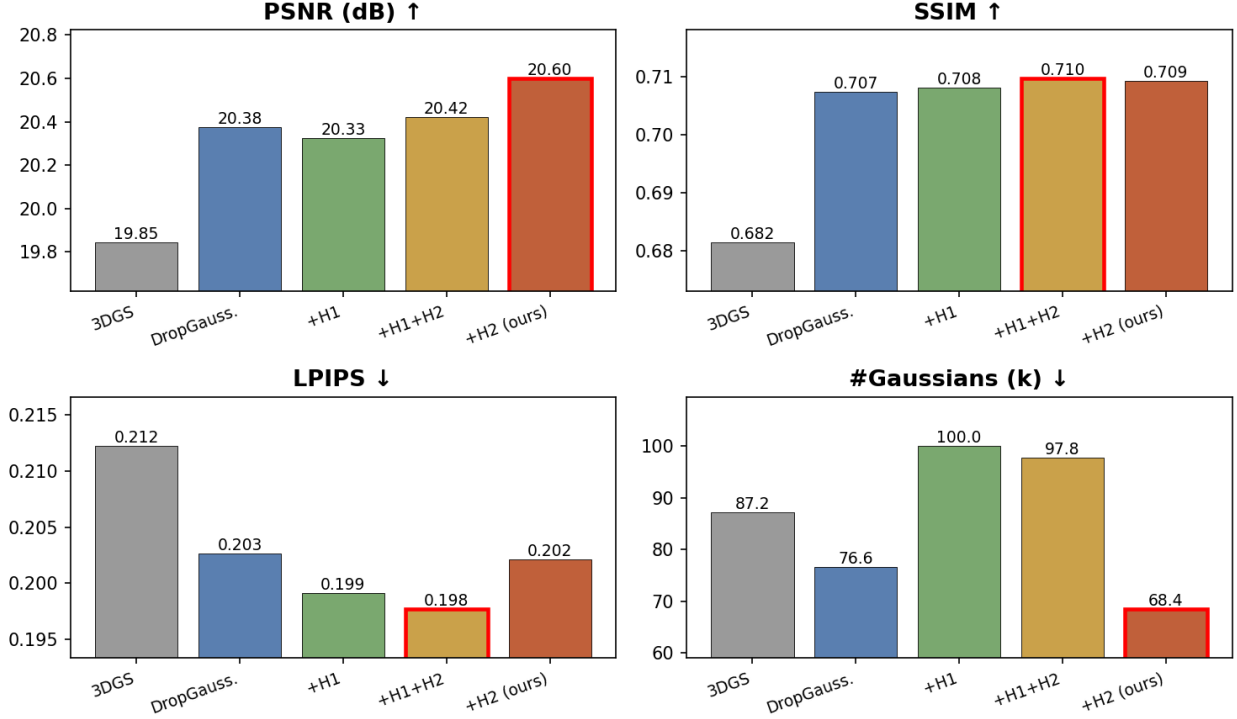


Fig. 2. Ablation summary over all 8 LLFF 3-view scenes (best per metric outlined in red). H2 attains the best PSNR and the most compact model; all variants beat 3DGS. H1/H1+H2 reach marginally better LPIPS/SSIM but inflate the Gaussian count.

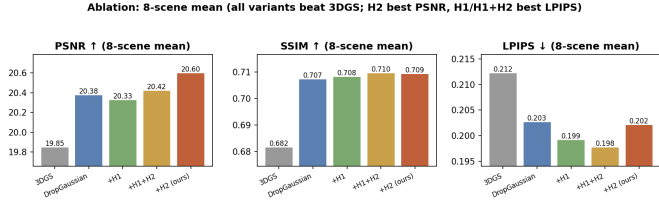


Fig. 3. 8-scene mean PSNR/SSIM/LPIPS for the five configurations. All variants beat 3DGS; H2 leads on PSNR while H1/H1+H2 are marginally ahead on the perceptual metrics.

H2 (mean SSIM 0.707 $\rightarrow$ 0.709, LPIPS 0.203 $\rightarrow$ 0.202), with scene-by-scene differences mostly within ± 0.003 . This is the expected signature of a sub-dB, compactness-oriented change: H2 removes redundant Gaussians without altering the perceived image, so it improves PSNR and model size while leaving the perceptual metrics flat—neither helping nor hurting them.

Overfitting (Fig. 6). The train/test PSNR gap falls from 18.2dB (3DGS) to 14.7dB (DropGaussian) to 14.0dB (+H2): H2 both raises test PSNR and lowers train PSNR, i.e. it regularizes slightly more than the original, which is consistent with producing a leaner model.

TABLE IV

Per-scene SSIM / LPIPS, original vs. +H2. Perceptual metrics are essentially tied (within ± 0.003 on most scenes), consistent with the sub-dB PSNR change.

Scene	SSIM ↑		LPIPS ↓	
	orig	+H2	orig	+H2
fern	0.755	0.754	0.188	0.191
flower	0.653	0.655	0.237	0.235
fortress	0.732	0.729	0.184	0.179
horns	0.674	0.682	0.256	0.259
leaves	0.650	0.649	0.200	0.203
orchids	0.512	0.512	0.255	0.256
room	0.854	0.855	0.158	0.154
trex	0.830	0.837	0.145	0.139
Mean	0.707	0.709	0.203	0.202

Noise floor (Fig. 8). Four identical original runs give PSNR std 0.07dB on fern but 0.25dB on room. The non-determinism is real and scene-dependent. H2's +0.22dB mean (paired $t \approx 2.3$, $p \approx 0.05$) is therefore only near-but-above this floor on PSNR alone—which is exactly why we treat the deterministic compactness result, not the PSNR, as the primary evidence. Fig. 7 plots the per-

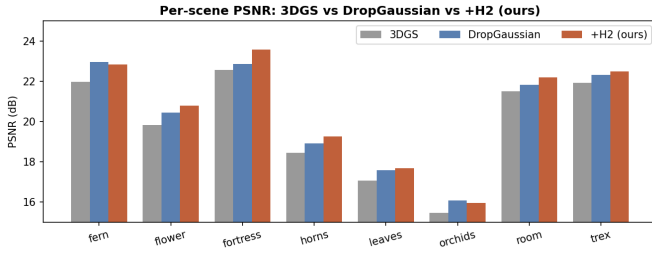


Fig. 4. Per-scene PSNR for 3DGS, DropGaussian and +H2. The $3DGS < DropGaussian \leq H2$ ordering holds on all eight scenes.

scene $\Delta PSNR$ against the fern noise band: the wins on fortress/room/horns clearly exceed it; the two small losses (fern, orchids) sit inside it.

Per-scene analysis (Fig. 4). Breaking PSNR down by scene clarifies where H2 helps. The largest gains are on fortress (+0.71), room (+0.37) and horns (+0.36)—scenes with substantial planar/low-frequency structure, where the original dropout’s gradient inflation spawns the most redundant floaters and H2’s more conservative densification therefore helps most (these three are also among the largest absolute Gaussian-count reductions). The gains shrink to near zero on the thin-structure scenes leaves (+0.09), where almost every Gaussian is genuinely needed, and turn marginally negative on fern and orchids (−0.13 each)—both within the single-scene noise band of Fig. 8. The $3DGS \rightarrow DropGaussian \rightarrow H2$ ordering, however, is monotonic on every scene, so the ranking itself is stable even where the H2-vs-original margin is noise-sized.

Efficiency. The $\sim 10\%$ fewer Gaussians translate into lower peak memory (e.g. fortress $124 \rightarrow 116$ MB, leaves $168 \rightarrow 153$ MB) and proportionally cheaper rendering. Training time is unchanged ($\sim 1:18$ vs $\sim 1:20$ per scene); H1 and H2 are both $O(N)$ element-wise and add no measurable overhead.

Qualitative (Fig. 9). On held-out views 3DGS shows the most artifacts; DropGaussian and H2 are visually close, as expected from a sub-dB difference. The improvement of H2 is therefore better characterized quantitatively (compactness, overfitting gap) than visually—a point we make explicitly rather than over-claim from a cherry-picked crop.

More qualitative results. Figs. 10–11 show the two indoor/structured scenes on which H2 helps most. Even here the three methods are perceptually close—again confirming that H2’s benefit is primarily a leaner, less overfit model rather than a visible jump in image quality. 3DGS still shows the most high-frequency floaters in background regions, which both DropGaussian and H2 suppress; H2’s renderings are produced by $\sim 10\%$ fewer Gaussians than DropGaussian’s, i.e. the same visual result at lower cost.

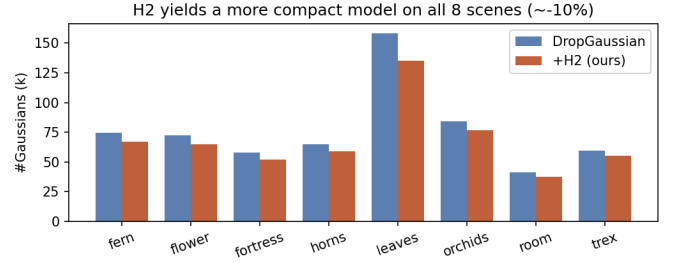


Fig. 5. Final Gaussian count, original vs. +H2, per scene. H2 is more compact on all eight scenes ($\sim 10\%$ mean)—a deterministic, noise-free effect.

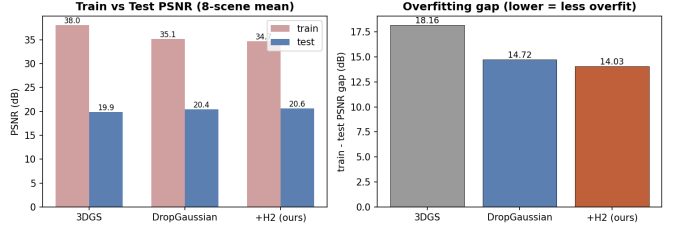


Fig. 6. Train vs. test PSNR (left) and the train/test gap (right), 8-scene mean. DropGaussian shrinks the gap over 3DGS; H2 shrinks it slightly further, i.e. it overfits less.

VII. Discussion & Conclusion

Did the hypotheses hold? H1: no. Opacity-aware dropout is -0.05 dB (wins 3/8) and harms H2 when combined. The reason is mechanistic: DropGaussian works precisely because it occasionally removes dominant high-opacity Gaussians, forcing the rest to compensate; H1 protects exactly those Gaussians and thereby blunts the regularizer. Tellingly, H1 also inflates the model to 100k Gaussians (vs. 76.6k original and 68.4k for H2)—the opposite of H2’s compactness—further evidence that biasing drops toward low-opacity Gaussians backfires. The bounded compensation needed to keep variance sane (Sec. V) cannot recover this loss. The hypothesis is cleanly falsified.

H2: yes, with a nuance. Drop-aware densification gives $+0.22$ dB (6/8) and, more robustly, a deterministic $\sim 10\%$ reduction in Gaussian count on every scene at equal-or-better SSIM/LPIPS, a smaller overfitting gap, and no extra cost. The PSNR gain alone sits near the rasterization noise floor, but the compactness and gap results are noise-free and confirm the L2 mechanism directly. We therefore regard H2 as a genuine, if modest, improvement: equal-or-better quality with a leaner, less-overfit model.

A quality-compactness trade-off. The ablation reveals a small Pareto structure rather than a single dominating configuration. H2 leads on PSNR, on model size, and on the overfitting gap; H1 and H1+H2 instead edge ahead on the perceptual metrics (LPIPS/SSIM) but at the price of 30–45% more Gaussians (100k/98k vs 68k). For sparse-view 3DGS—where deployment cost and floater suppres-

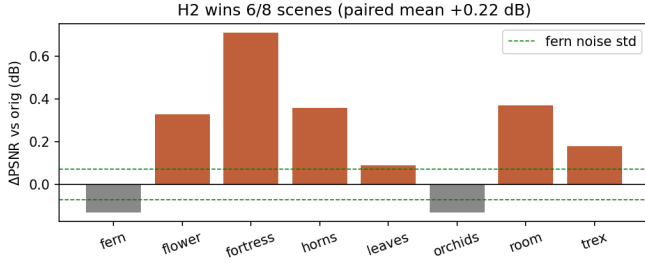


Fig. 7. Per-scene Δ PSNR of H2 vs. original. Dashed band: $\pm$ fern noise std. H2 wins 6/8; the two small losses lie within the noise band.

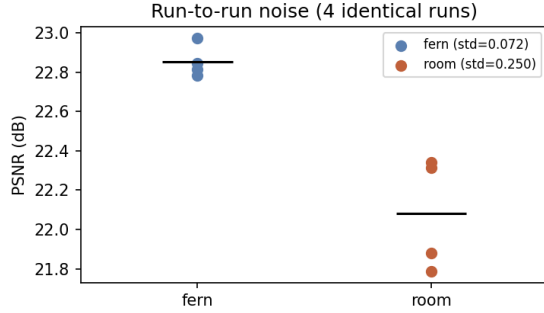


Fig. 8. Run-to-run PSNR over four identical original runs. Non-deterministic rasterization gives std 0.07 dB (fern) and 0.25 dB (room).

sion are first-class concerns—we view H2’s operating point (best fidelity at the smallest model) as the more useful one, but the trade-off makes the “best” configuration genuinely application-dependent, and we report both rather than declaring a single winner.

Why the gains are small. DropGaussian is already a strong, carefully-tuned regularizer. Refining the shape of the dropout (H1) or its coupling to densification (H2) operates at the margin and is easily masked by stochastic rasterization noise. This is an honest near-zero/small result; we deliberately avoid cherry-picking a single favorable scene, which the noise analysis (Fig. 8) shows would be misleading. We consider exposing this noise floor itself a useful contribution: several sparse-view papers report single-run, single-scene deltas of a few tenths of a dB, which our measurements suggest can be within run-to-run variance.

Limitations & future work. Our study is confined to LLFF 3-view; the noise floor would ideally be estimated per configuration with more repeats, and the analysis extended to Mip-NeRF360 and to 6/9-view settings. More importantly, the results suggest that larger, structured interventions are needed to move the needle: (i) DropBlock-style spatial dropout that removes correlated clusters of Gaussians rather than i.i.d. singletons, directly attacking L1’s “routing-around” failure; and (ii) overfitting-adaptive drop schedules that allocate more regularization to the

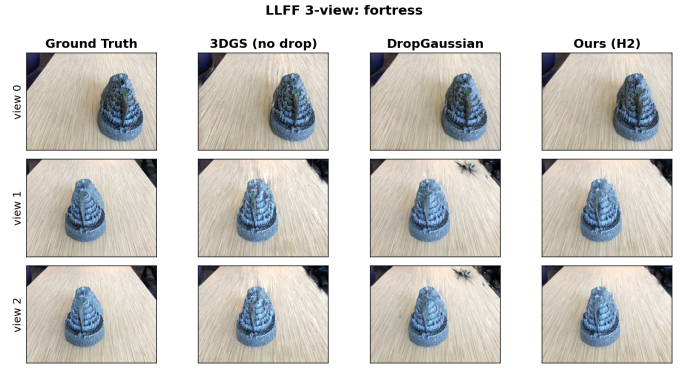


Fig. 9. Held-out views (fortress): GT, 3DGS (no drop), DropGaussian, Ours (H2). Differences are subtle at this PSNR scale, consistent with the quantitative findings; more scenes (horns, room) are in the data package.

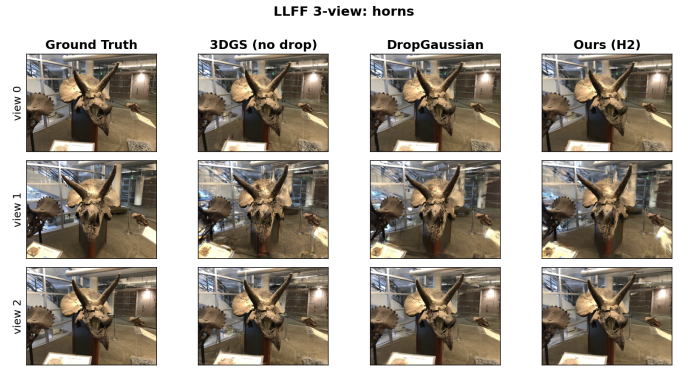


Fig. 10. Held-out views (horns, where H2 wins +0.36 dB): GT, 3DGS, DropGaussian, Ours (H2).

early densification phase, when redundant Gaussians are born. Finally, H2 suggests a broader principle worth testing elsewhere: any regularizer that alters the forward pass should re-examine its interaction with adaptive density control, since that controller was tuned for the unperturbed pass.

AI Usage Statement

AI tools (Claude) were used to assist with Docker environment setup, debugging, the code implementation of H1/H2, experiment orchestration/scripting, figure generation, and the drafting/polishing of this report. All experiments were executed by the author on the described hardware, and every reported number is a real measurement from those runs.

References

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Trans. Graphics*, vol. 42, no. 4, 2023.
- [2] H. Park, G. Ryu, and W. Kim, “DropGaussian: Structural Regularization for Sparse-view Gaussian Splatting,” in *Proc. IEEE/CVF CVPR*, 2025.
- [3] Z. Zhu, Z. Fan, Y. Jiang, and Z. Wang, “FSGS: Real-Time Few-Shot View Synthesis using Gaussian Splatting,” in *Proc. ECCV*, 2024.

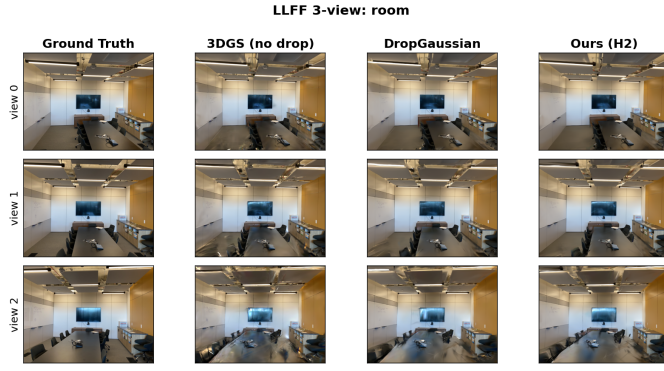


Fig. 11. Held-out views (room, where H2 wins +0.37 dB): GT, 3DGS, DropGaussian, Ours (H2).

- [4] B. Mildenhall, P. Srinivasan, R. Ortiz-Cayon et al., “Local Light Field Fusion: Practical View Synthesis with Prescriptive Sampling Guidelines,” *ACM Trans. Graphics*, vol. 38, no. 4, 2019.
- [5] A. Jain, M. Tancik, and P. Abbeel, “Putting NeRF on a Diet: Semantically Consistent Few-Shot View Synthesis,” in *Proc. ICCV*, 2021.
- [6] M. Niemeyer, J. T. Barron, B. Mildenhall et al., “RegNeRF: Regularizing Neural Radiance Fields for View Synthesis from Sparse Inputs,” in *Proc. IEEE/CVF CVPR*, 2022.
- [7] J. Yang, M. Pavone, and Y. Wang, “FreeNeRF: Improving Few-shot Neural Rendering with Free Frequency Regularization,” in *Proc. IEEE/CVF CVPR*, 2023.